Eclipse Attack Simulator - Final Report

As of 2026-10-28

We build a discrete-event simulator of Ethereum's discv5 Kademlia-style peer discovery, reproduce the Marcus-Heilman-Goldberg low-resource eclipse attack (ePrint 2018/236), evaluate the paper's countermeasures that fit our model, and introduce a new identity-churn peer-scoring defence that in simulation resists the attack up to 512 attacker IPs at a near-zero honest false-positive rate - dominating the paper's per-/24 cap on the security-vs-false-positive tradeoff.

Background

An eclipse attack isolates a victim node by occupying all of its peer connections with attacker-controlled peers, so the victim sees only the attacker's view of the chain. In Ethereum this enables attacks such as splitting mining power, suppressing or delaying transactions and blocks, and setting up double spends.

Ethereum discovers peers using discv5 (previously discv4), a Kademlia-style protocol. Each node has a 256-bit node ID and keeps a routing table of 256 *k-buckets* indexed by the log-distance (the position of the highest differing bit) between its own ID and each known peer's ID. Outbound connections are drawn from this table, so an attacker who controls the table controls who the victim talks to. Marcus, Heilman & Goldberg (ePrint 2018/236) showed this table could be captured with very few resources - the *low-resource* eclipse attack.

System model and simulator

We implemented a discrete-event simulator: a time-ordered event heap driving three event types.

1. **Honest churn** - each honest node toggles online/offline on exponential timers.
2. **Victim discovery** - the victim periodically learns an honest node and tries to insert it into its table.
3. **Adversarial flood** - the attacker injects sybil nodes at a high rate once the attack begins.

The network has ~800 honest nodes, a fraction of them hosted in shared cloud /24 subnets to model realistic IP clustering. The attacker controls A /24 subnets, each holding many IPs. The attack starts after the table has bootstrapped. We measure the adversary's share of the victim's routing table at the end of the run; because outbound peers are drawn from the table, this share is our eclipse metric, and *eclipse success* means the adversary holds at least 75% of the table.

Routing table (implemented ourselves)

As the brief requires, we implement the routing-table logic ourselves - that is where the vulnerability lives. A peer is placed in bucket `logdist(victim, id)`, and each bucket holds k = 16 entries.

- **Endpoint verification:** a node is admitted only if it answers a ping (modelled as being online). Attacker nodes are always online.
- **Eviction:** on a full bucket the least-recently-seen entry is pinged; if it answers it is kept and the newcomer is dropped, so a bucket of live honest nodes resists takeover. The attacker gains a slot only when an honest incumbent has churned offline.

This reproduces the exact property the attack must overcome and makes honest churn the lever the attacker exploits.

The low-resource eclipse attack

Node IDs are cheap to generate (any key pair), and pre-fix clients placed no limit on how many node IDs a single IP could advertise. The attacker therefore floods the victim with many sybil node IDs from a small number of machines; as honest nodes churn out, attacker nodes fill the freed slots until they dominate the table - and therefore the victim's outbound peers.

We reproduce this directly: during the attack window the attacker emits sybil nodes with random IDs (so they spread across buckets exactly like honest nodes) from its A /24 subnets at a high rate. We then sweep the attacker's resources and measure how large it must be to eclipse the victim.

Experiment 1 - attack vs attacker resources

The attacker holds A /24 subnets of 16 IPs each; the x-axis is total attacker IPs; success = adversary holds >=75% of the table, over 30 trials per point. Values are eclipse success probability.

Attacker IPs	No defence	One key per IP	Per-/24 cap (geth)	Identity-churn (ours)
16	1.00	0.00	0.00	0.00
32	1.00	0.00	0.00	0.00
64	1.00	0.00	0.00	0.00
128	1.00	1.00	0.00	0.00
256	1.00	1.00	1.00	0.00
512	1.00	1.00	1.00	0.00

With no defence the victim is eclipsed by an attacker with just 16 IPs - the low-resource result. The defences form a clear strength ladder, detailed in the next sections. (Figure: `success_prob.png`.)

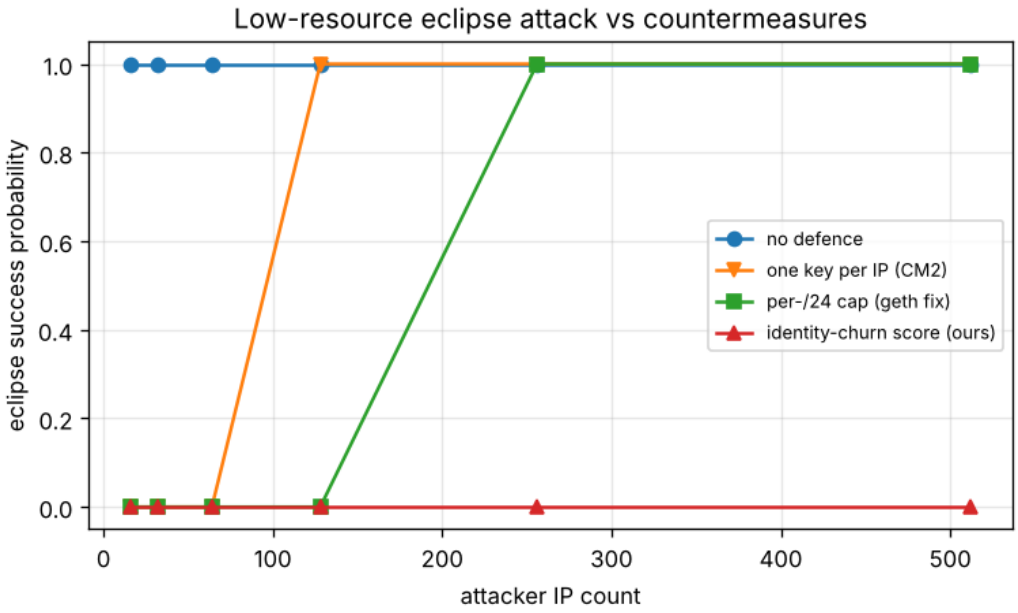


Figure 1. Eclipse success probability vs attacker IP count.

The paper's seven countermeasures

The paper proposes seven countermeasures (Sections III, V and VI). One lives at the routing-table layer our simulator models and is implemented and evaluated; one is assumed by our metric; the remaining five act at layers we abstract away or target a different attack, and are documented with reasons. (geth v1.8.0 fully adopted CM1, CM5 and CM6, adopted CM2 only partially as the /24 limit, and did not adopt CM3, CM4 or CM7.)

#	Countermeasure	In our simulator
CM1	Cap incoming TCP connections (rely on outbound)	**Assumed** - our metric is the adversary's share of the table that feeds outbound selection
CM2	One key per IP + geth's /24 limit (2/bucket, 10/table)	**Implemented & evaluated**, as two mechanisms: strict one-key-per-IP and the per-/24 cap
CM3	Secret node-ID -> bucket mapping	**Not evaluated** - defends ID-grinding into targeted buckets, orthogonal to our volume flood
CM4	Secret self-lookup	**Not evaluated** - same reason
CM5	Aggressive seeding on restart	**Not modelled** - no reboot/seed phase in our steady-state model
CM6	Close the reboot exploitation window	**Not modelled** - same reason
CM7	Use nonces instead of UDP timestamps	**Not applicable** - fixes the separate time/clock-manipulation attack (Section VI), not the table takeover we simulate

So of the seven: **1 implemented & evaluated (CM2), 1 assumed (CM1), 5 not evaluated (CM3-CM7).** Experiment 1 shows the strength ladder: no defence fails

at 16 IPs; one-key-per-IP at ~128 IPs (IP limits are insufficient because a low-resource attacker rents a whole /24 of cheap IPs - exactly why the paper moved to /24 limits); the per-/24 cap at ~256 IPs; ours never, up to 512.

Our new defence - identity-churn peer scoring

The per-/24 cap is a *static concurrency* rule: it caps how many nodes from a subnet sit in the table at once. It cannot distinguish an honest cloud /24 that hosts a handful of stable nodes from a /24 through which a low-resource attacker is cycling hundreds of distinct sybil IDs.

Our defence keys on exactly that difference. For each /24 origin we track the number of **distinct node IDs ever advertised** from it. An honest /24 shows few, stable IDs; a sybil /24 shows a flood. A subnet whose distinct-ID count exceeds a threshold T is treated as a sybil source: its nodes are refused admission and are the first to be evicted, while low-churn origins are kept. Enforcement acts only under capacity pressure, so honest nodes are never rejected while a bucket still has room. This is our own rule - not libp2p's peer-scoring module.

Experiment 2 - defence evaluation and false positives

We measure the honest **false-positive rate** (honest nodes wrongly excluded while the bucket still had room) under honest churn, against **attack severity** (mean adversary table share) at a fixed attacker of 16 subnets / 256 IPs. Lower-left is better.

Per-/24 cap (geth fix), varying the limit L:

L	False-positive rate	Adversary table share
1	0.026	0.70
2	0.002	0.95
3	0.000	0.96
4	0.000	0.96
6	0.000	0.96

Identity-churn score (ours), varying the threshold T:

T	False-positive rate	Adversary table share
10	0.002	0.18
20	0.000	0.27
40	0.000	0.35
80	0.000	0.45
160	0.000	0.51
320	0.000	0.63

The /24 cap can only reach near-zero false positives by letting the attacker take ~95% of the table. Ours holds the attacker to 18-27% at a false-positive rate at or near zero - it dominates the cap across the whole curve. (Figure: `defence_tradeoff.png`.)

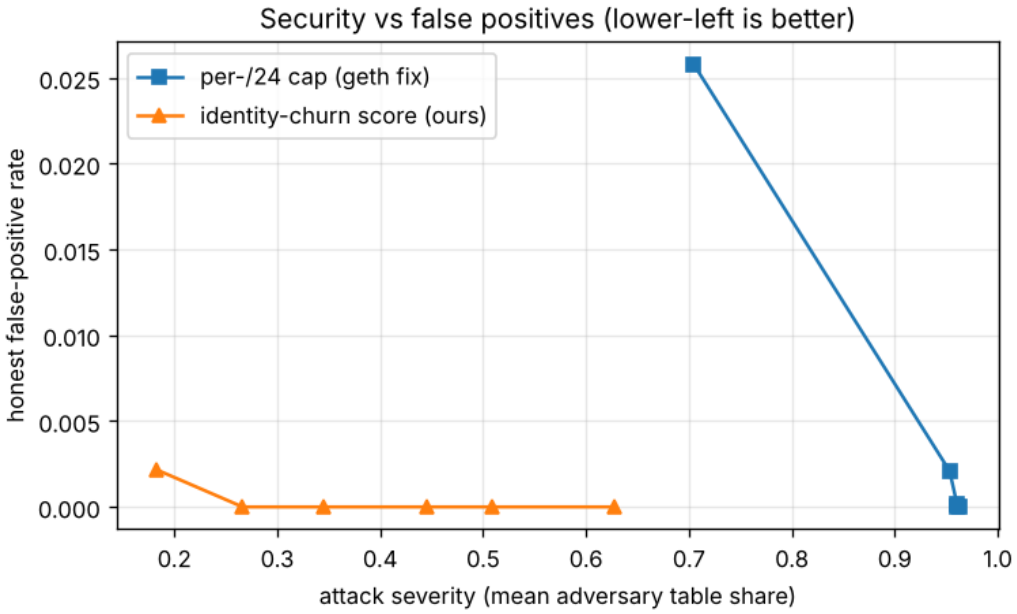


Figure 2. Security vs false positives (lower-left is better).

What is new, and limitations

New vs the paper:

- The paper proposes countermeasures but reports no false-positive evaluation under honest churn. We measure it and expose the /24 cap's hidden cost on clustered honest (cloud-hosted) peers.
- Our defence keys on identity churn *per network origin* (a sybil-rate signal) rather than a static per-/24 concurrency cap, and in simulation achieves both lower attack severity and lower false positives.

Limitations:

- Our defence assumes the attacker reuses a bounded set of /24s and must emit many IDs per origin - true for the low-resource attack. An attacker with very many /24s, or one that injects IDs slowly, evades the signal, but is then no longer low-resource (the paper's threat model).
- CM3-CM6 target layers (ID/lookup secrecy, reboot seeding) outside our discovery-table abstraction; testing them would need a connection/reboot model.

Reproducing the results

Files (in `eclipse_sim/`): `kademlia.py` (routing table + all defence modes), `simulator.py` (discrete-event engine, churn, adversary), `experiments.py` (both experiments and the plots), `README.md`.

```
pip install matplotlib
python experiments.py
```

Prints both result tables and saves `success_prob.png` and

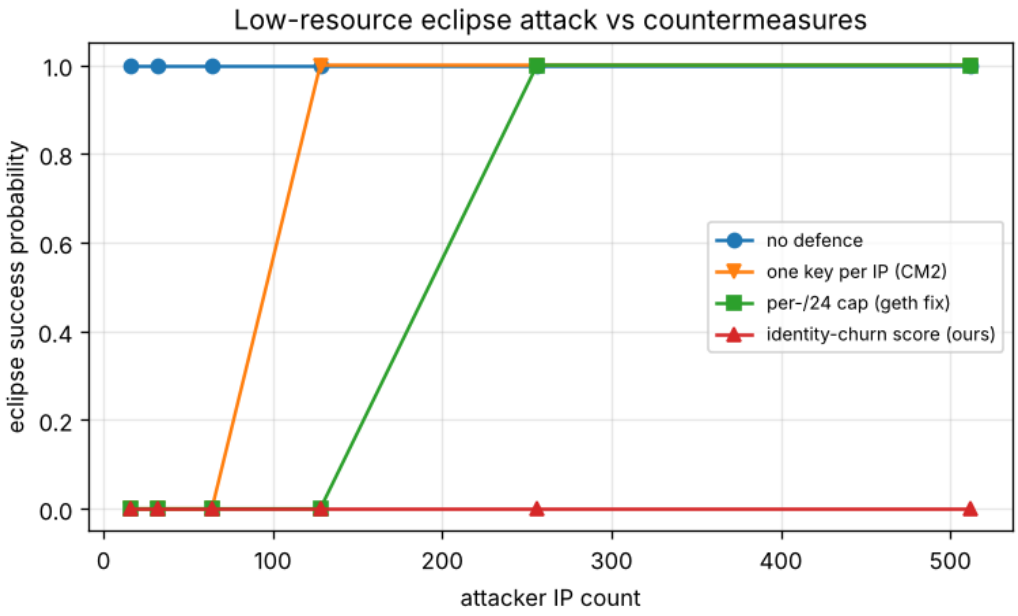


Figure 1. Eclipse success probability vs attacker IP count.

`defence_tradeoff.png` in about 50 seconds.

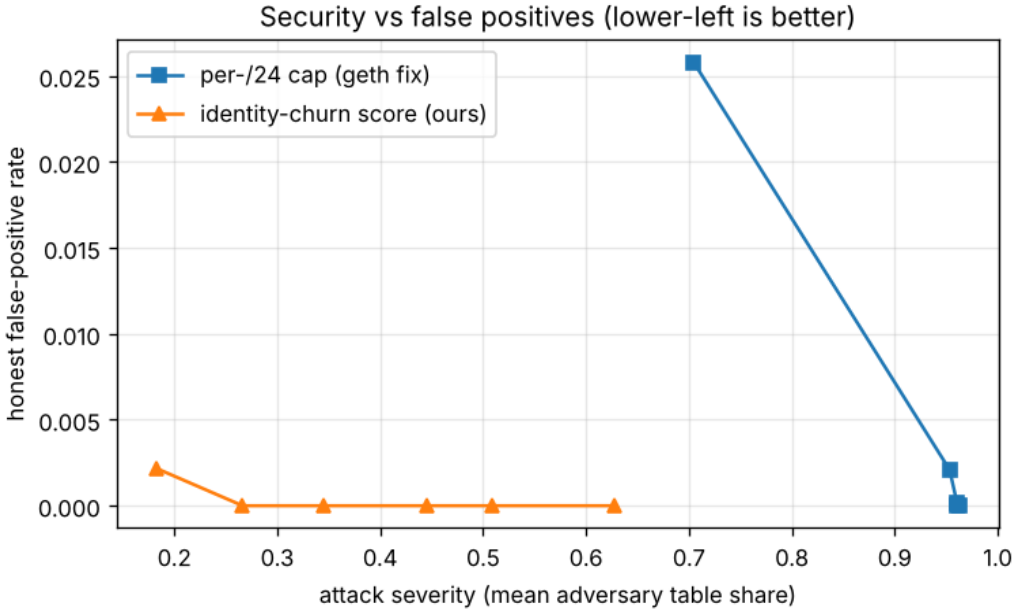


Figure 2. Security vs false positives (lower-left is better).

References

- Marcus, Heilman & Goldberg. *Low-Resource Eclipse Attacks on Ethereum's P2P Network.* ePrint 2018/236. https://eprint.iacr.org/2018/236
- Heilman, Kendler, Zohar & Goldberg. *Eclipse Attacks on Bitcoin's P2P Network.* USENIX Security 2015.
- Henningsen, Teunis, Florian & Scheuermann. *Eclipsing Ethereum Peers with False Friends.* EuroS&PW 2019. https://arxiv.org/abs/1908.10141